Final Year Project Proposal

Hostel Management System

Developed By:

Ms.Insha Murtaza (Enrollment BS CS SP-2023-Eve-0212)

Supervised By
Ms.Shamsa Bano

Project Coordinator
Dr. Fakhra Kashif

Department of Computer Science
Federal Urdu University of Arts, Science & Technology, Islamabad
SESSION [2023-2027]

A Project Presented to

Federal Urdu University of Arts, Science and Technology

In Partial Fulfillment of the Requirement for the Degree of

Bachelor of Computer Science

(Software Systems)

By

Ms.Insha Murtaza

Student Signature: _____

Date of Submission: _____

Federal Urdu University of Arts, Science and Technology, Islamabad

Plagiarism Certificate

This is to certify that I, Ms. Insha Murtaza (Enrollment BS CS SP-2023-Eve-0212), a student of the Department of Computer Science, Federal Urdu University of Arts, Science and Technology, Islamabad, declare that our Final Year Project proposal has been checked by our supervisor, and the similarity index is less than 20%, which is within the acceptable limit set by HEC. The report is attached here with as Appendix A.

Date: _____

Mr./Ms. Student Name: _____
Signature: _____

Dr./Mr./Ms. Supervisor Name:

_____

Designation:

_____

Signature:

_____

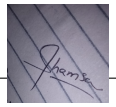
Supervisor Section

Supervisor Comments:

- Proposal Idea is good enough to take it as a Final Year Project: ☐ Yes ☐ No
- Project for real client (if yes, attach evidence letter in proposal defense presentation): ☐ Yes ☐ No

Comments (if any):

Supervisor Name: Ms. Shamsa Bano

Signature with Date: 

Submission Date: _____

Evaluator Section

Evaluator Comments:

- Proposal Idea is good enough to take it as a Final Year Project: ☐ Yes ☐ No
- Project for real client (if yes, attach evidence letter in proposal defense presentation): ☐ Yes ☐ No

Evaluator Comments (if any):

Evaluator Name: _____

Signature with Date: _____

Project Proposal Status: Approved / Not Approved

Revision History

Version	Description	Author(s)	Date
1.0	Initial Release	Ms. Insha Murtaza	06/04/2026

Contents

Supervisor Section **2**

Evaluator Section **3**

Revision History **4**

1 Abstract **9**

Abstract **9**

2 Introduction **10**
2.1 Background 10
2.2 Problem Statement 10
2.3 Aims and Objectives 11
2.4 Scope of the Project 11
2.5 Modules Description 12
2.5.1 Login Management 12
2.5.2 Room Management Module 12
2.5.3 Complaint Management Module 12
2.5.4 Student Management Module 12
2.5.5 Account Management Module 13
2.5.6 Inventory Management Module 13
2.5.7 Facility Management Module 13
2.5.8 Modules and Sub-Modules 13
2.6 Significance 14
2.7 Methodology 15

3 Literature Review **16**
3.1 Existing System 16
3.2 Proposed System 16
3.3 Comparative Analysis 16

4 Requirement Analysis **18**
4.1 Introduction 18
4.2 Purpose 18
4.3 Scope 18
4.4 Overall Description 18
4.5 Requirement Specification 19
4.5.1 Functional Requirements 19

4.5.2 Non-Functional Requirements 20
4.6 Software Requirements 20
4.7 Database Requirement 20
4.8 Actor Identification 20
4.9 Database Schema 21

5 System Analysis and Design **23**
5.1 Gantt Chart 23
5.2 Process Model and System Design Approach 23
5.3 Use Case Diagram 24
5.4 Fully-Dressed Use Cases 25
5.5 Module Use Case Diagram 30
5.6 Class Diagram 33
5.7 Sequence Diagram 34
5.8 ER Diagram 35
5.9 Domain Model 36
5.10 Collaboration Diagram 37
5.11 Activity Diagram 38

6 Implementation **39**
6.1 Development Tools 39
6.2 Front End Implementation 39
6.3 Back End Implementation 39
6.4 Database Implementation 39

7 Future Work and Conclusion **40**
7.1 Future Enhancements 40
7.2 Conclusion 40

List of Tables

5.1 Project Gantt Chart 23
5.2 Use Case Specification for Register Account 25
5.3 Use Case Specification for Login Management 25
5.4 Use Case Specification for Room Management 26
5.5 Use Case Specification for Complaint Management 27
5.6 Use Case Specification for Payment Management 27
5.7 Use Case Specification for Student Management 28
5.8 Use Case Specification for Account Management 28
5.9 Use Case Specification for Inventory Management 29
5.10 Use Case Specification for Facility Management 29

List of Figures

5.1 Hostel Management System 24
5.2 Login Management 30
5.3 Room Management 30
5.4 Complaint Management 30
5.5 Payment Management 31
5.6 Student Management 31
5.7 Complaint Management 31
5.8 Inventory Management 32
5.9 Facility Managements 32
5.10 Class Diagram 33
5.11 Sequence Diagram 34
5.12 ER Diagram 35
5.13 Domain Model 36
5.14 Collaboration Diagram 37
5.15 Activity Diagram 38

Chapter 1

Abstract

In today's busy student life managing a hostel can be difficult for both students and the hostel administration.Our web-based Hostel Management System is designed to make Hostel Management easier, more organized, and more convenient for students and employees. The Web-based Hostel Management system allows students to register to book rooms, pay their fees,and check their attendance through a simple and easy-to-use interface.However, employees can use a dashboard to manage rooms, handle mess schedules, and generate reports quickly according to the requirement. One of the main features of this system is the Smart Complaint System.Students can easily submit their issues like Wi-fi ,cleanliness, maintenance and other issues.The system organizes these complaints into categories,shows which problems are more urgent, and notifies the employee.Students can also check the status of their complaints,which makes the process faster and easier to understand. The second important feature is the Room Swap Request System.Students who want to change or change their room must send a request with a valid reason.The system helps find suitable options and allows both students and employees to approve the request .When the employee approves the request,the system update automatically saves time and reduces the manual work of hostel management. In general,this system reduces manual work,improves communication ,and provides a better hostel experience.It is a modern solution that makes hostel management more efficient and simple and easier.

Chapter 2

Introduction

2.1 Background

The Hostel Management System will be a web-based platform designed to simplify hostel operations,maintain student records properly, help students and hostel staff manage hostel activities easily.Key features of the system include handling room allocation,student registration,fee management ,complaint handling,and maintaining hostel records securely. The system will help students ,employees,and administrators communicate easily,check updates,and share information quickly.

Many hostels still depend on manual and paper work, making managing student records,room details, and hostel activities slow and time consuming.It is often difficult for hostel administration to manage student information ,room availability ,fee records , complaints, and other hostel-related tasks efficiently.In today's modern world, a digital hostel management system is very important to improve organization and reduce workload.

Most existing manual systems are not efficient because they increase the chances of data loss,delay in communication, and errors.Students also face difficulties in checking room details, submitting complaints and quickly accessing hostel-related information. The proposed Hostel Management System will provide a simple, and interactive environment for both hotel staff and students.

This web-based system will be developed to manage hostel tasks in an easy and more organized way to help administrators and employees complete their work accurately and faster.The system will also improve communication by allowing students to easily submit complaints, request facilities,view hostel updates online and can also check fee status .The main goal of this system is to make hostel management faster,reduce paperwork, improve record keeping, safer,and support hostel staff in performing their tasks efficiently.

2.2 Problem Statement

Many hostels still use manual systems to manage student records,fee details complaints, hostel facilities, and room allocation.These traditional methods are time consuming,difficult to manage and increase chances of data loss and errors.Hostel staff often face problems in maintaining accurate records,while students experience delays in getting information related to fees,complaints,rooms and hostel services.

Another major issue is that existing hostel management system are often complicated

for students and non-technical staff members.Because of this,many users feel confused while using the system and communication between hostel administration and students becomes less effective. Manual paperwork also make it difficult to track hostel activities properly and reduces overall efficiency.

Students living in hostels may also face problems in submitting complaints,requesting facilities and checking updates quickly.Delays in handling complaints and maintaining records can create management issues and dissatisfaction among students.Therefore,there is a strong need for a hostel management system that is simple,secure, help reduce paperwork , maintain records,improve communication,and make hostel management more organized and effective.

2.3 Aims and Objectives

The main objective of this project will be to develop a simple and efficient Hostel Management System that helps students and hostel staff manage hostel activities easily.The system aims to provide a user-friendly environment for managing student records,fee management, hostel facilities,room allocation, complaint submission in an organized way.

Another important objective is to reduces manual paperwork and improve the accuracy of hostel records by storing all information securely and digitally.The System will also focus on improving communication between employees,students and administrators by allowing students to submit complaints,check updates online and request complaints.

The project also aims to save time and increase management efficiency by automating different hostel operations.Students will be able to access important hostel information easily and quickly, while administrators can monitor hostel activities in a better way.Overall, the system will be designed to reduce management difficulties, improve record keeping, make hostel operations faster, increase transparency,safer and more effective.

2.4 Scope of the Project

The proposed Hostel Management System is designed to provide a digital platform for managing hostel services efficiently.

1. Login Management
2. Hostel Management
3. Hostellite Management
4. User Management
5. Room Management
6. Complaint Management
7. Booking Management
8. Fee Management
9. Facilities Management

10. Request Management
11. Room Allocation
12. Complaint Tracking
13. Online Fee Payment

2.5 Modules Description

The system will be divided into several modules that will manage Students, employee and hostel operations in an organized manner.

2.5.1 Login Management

This module handles user authentication by allowing users to securely login and logout.

2.5.2 Room Management Module

This module covers the core lifecycle of room allocations, physical bed availability trackers, and structural mappings, implementing the **Room Management** use case.

- **Student Component:** Allows hostellites to view architectural room configurations, check real-time bed vacancies, and formally submit room reservation applications online.
- **employee Component:** Empowers the employee to review pending room requests, match student profiles against empty slots, and execute final administrative approval or rejection for room allocations.

2.5.3 Complaint Management Module

Acting as a direct operations bridge for issues, maintenance, and resident requests, this module maps directly to the **Complaint Management** use case.

- **Student Component:** Allows students to lodge technical or utility grievances (such as electrical, plumbing, or Wi-Fi faults) and track the system-generated ticket.
- **employee Component:** Enables the employee to monitor incoming issues on an administrative control panel, assign technical specialists, update resolution status tags (e.g., Pending, In-Progress, Closed), and append final action notes.

2.5.4 Student Management Module

This module handles institutional profiles, baseline registries, and occupancy parameters for the hostel residents, corresponding to the **Student Management** use case.

- **Student Component:** Provides read/write pathways for students to update their emergency guardian contact cards, medical detail logs, and personal details.
- **employee Component:** Grants the employee full administrative control to inspect comprehensive student rosters, filter resident databases by room allocations, track active occupancy metrics, and modify disciplinary or check-in tags.

2.5.5 Account Management Module

Focused on baseline system settings, profile configurations, and cryptographic access parameters, this module maps to the **Account Management** use case. It allows both Students and employees to manipulate their user-profile settings, manage multi-factor parameters, and execute secure password resets. The system verifies identity via old cryptographic keys before committing adjustments to active relational database tables.

2.5.6 Inventory Management Module

This module handles hardware tracking, logistics balances, and asset inventories across the building blocks, mapping exclusively to the **Inventory Management** use case. Restricted to administrative personnel (**employee**), it enables real-time logging of physical assets (such as furniture, fans, and appliances), monitors stock counts for the mess or common rooms, and flags broken/depleted items needing urgent replacement.

2.5.7 Facility Management Module

This module controls all the shared hostel facilities and manages the entire facility management process.

- **Student Component:** Gives students a real-time view of shared service parameters, operational hours, and availability calendars for zones like the Gym, Mess hall, or Laundry room.
- **Employee Component:**It enables management staff to manage hostel timing, update facility status and plan regular inspection

2.5.8 Modules and Sub-Modules

Payment Management

1. Generate Fee Challan
2. Submit Payment (Upload Receipt)
3. Verify Payment Status
4. Track Payment History

Room Management

1. Check Room Availability
2. Submit Room Request
3. Allocate Room
4. Update Room Status

Complaint Management

1. Submit Complaint

2. Track Complaint Status
3. Resolve Complaint

Login Management

1. Sign In
2. Sign Out
3. Forget Password

Student Management

1. Student Registration
2. View Student Profiles
3. Manage Student Records

Account Management

1. Create Account
2. Update Account Credentials
3. Manage User Roles

Inventory Management

1. Track Hostel Assets
2. Update Stock Count
3. Monitor Asset Condition

Facility Management

1. Add/Update Hostel Facilities
2. Toggle Facility Visibility States

2.6 Significance

The Hostel Management System will play an important role in improving hostel operations and management activities.It will help employees,students records, handle complaints, manage room allocations, and maintain hostel information in a more efficient and secure way.

The system will reduce the use of manual work and minimize time consumption by automating common hostel management processes.It will also improve communication between employees and students by providing quick access to hostel updates, fee details, complaints and other important information.

For students, the system will increase convenience and transparency,as they will be able to check room details, request facilities,view fee status,submit complaints without frequent visit to the hostel office.It will also reduce the chances of errors that usually occurs in manual records keeping in paper-based management system.

Overall,the Hostel Management System will improve productivity, increase data security and make hostel operations faster, more reliable, easier for students and employees.

2.7 Methodology

The System follows a structured web-based development approach:

1. **Requirement Gathering:** The hostel requirements are collected by analyzing hostel operations.room allocation procedures, complaints handling , fee management process and other student problems.

2. **System Design:** The system architecture, UML diagrams, user interface, database structures are designed according to hostel requirements.

3. **Development:** Front-end technologies such as HTML,CSS,JavaScript,and Bootstrap are used for user interface development, while PHP/Python and MySQL are used for backend and database implementation.

4. **Testing:** The system is tested to identify errors, improve performance, and ensure secure and accurate hostel management operations.

5. **Deployment:** The final Hostel Management System is deployed on a local or web server for practical implementation and future enhancements.

Chapter 3

Literature Review

3.1 Existing System

In the field of Hostel Management, several traditional methods are currently being used to manage student records, complaints, fee details and hostel documentation.Most hostels still rely on manual record-keeping or basic digital tools such as simple database software and spreadsheets.

In manual systems, all student information, fee details, complaints,room records are stored in physical files and registers.This approach makes it difficult to search,maintain, and update records efficiently.It also increases the risk of data duplication,data loss, and mismanagement of important hostel information.

Some hostels use basic digital systems that allow for simple data entry and storage.However, these systems often lack advanced features such as automated complaint tracking, secure user authentication ,online fee management ,room allocation,and real-time notifications. As a result, hostel employees may still face difficulties in managing hostel operations effectively.

In general, existing systems provide only limited support for hostel management activities, do not fully address the need for a centralized,secure, and efficient platform to manage students,rooms, fees, complaints, and hostel records.

3.2 Proposed System

The proposed Hostel Management System will provide an online platform for managing residential services and hostel-related activities.Students will be able to create accounts, submit room booking requests,view allocation statuses, and track complaints through the system.employee will manage room information,store student registration documents,and communicate with resident using a centralized interface.The system will support efficient information management, improve accessibility, reduce manual work, and enhance the overall effectiveness of hostel service delivery.

3.3 Comparative Analysis

The existing system mainly depends on manual record keeping and traditional communication methods,which can keep track of residents, make room allocation and retrieval time consuming.Managing large number of student profiles, room records and fee receipts may

also increase the chance of errors and delays.In comparison, the proposed Hostel Management System provides a centralized online platform where students and administrators can manage fee collections, room allotments. and complaints efficiently.This improves accessibility, reduces the paperwork,and enhances the overall effectiveness of residential facility management.

Chapter 4

Requirement Analysis

4.1 Introduction

The Software Requirement Specification (SRS) document will describes the functional and non-functional requirements of the Hostel Management System.This system will be developed to digitize hostel services and improve accessibility for students and hostel administration.

4.2 Purpose

The purpose of this system will be develop a secure and efficient web-based platform that will manage hostel activities digitally.

4.3 Scope

The system enables students to register, submit complaints,book rooms and manage hostel-related requests.employees can manage hostel data,students and financial records. The system consists of multiple core modules including hostel management, user management, booking , fee,facility and room allocation management.

4.4 Overall Description

The proposed Hostel Management System is designed to simplify hostel administration tasks,improve communication between students and employees ,enhance data security and create a more efficient workflow for managing hostel operations.

The front-end of the system will be developed using HTML,CSS,and JavaScript to create an interactive and user-friendly interface for students for administration.

The backend will be developed using will be using PHP to handle system functionality and server-side operations such as login, complaint handling ,fee management,booking requests, room allocation.

MySQL will be used as the database to securely store and manage student records, complaints, fee records and hostel data.

4.5 Requirement Specification

4.5.1 Functional Requirements

1. **Security Management**
   (a) Sign In
   (b) Sign Out
   (c) Forget Password
2. **User Management**
   (a) User registration and role assignment (Student, employee, Admin)
   (b) Account Management and profile updates
3. **Hostel Management**
   (a) Add, update, and delete hostel information
   (b) Manage hostel blocks and capacity
4. **Room Management**
   (a) Add and manage rooms
   (b) Track room availability and occupancy
5. **Booking Management**
   (a) Room booking by students
   (b) Approval/Rejection by admin or employee
   (c) Booking history tracking
6. **Fee Management**
   (a) Fee generation and management
   (b) Payment tracking
   (c) Due fee alerts
7. **Complaint Management**
   (a) Submit complaints by students
   (b) Track complaint status
   (c) Complaint resolved by employee
8. **Facility Management**
   (a) Manage hostel facilities (mess, electricity, and laundry etc)
   (b) Facility availability updates
9. **Request Management**
   (a) Handle student requests(leave, change room etc)
   (b) Approval workflow for requests

4.5.2 Non-Functional Requirements

1. **Performance:** System will respond quickly and handle multiple users.
2. **Security:** The system will ensure encrypted passwords and secure login.
3. **Usability:** Simple and user-friendly interface.
4. **Availability:** 99% system uptime.
5. **Scalability:** Support multiple hostels and large number of students.

4.6 Software Requirements

1. **Frontend:** HTML, CSS, JavaScript, Bootstrap
2. **Backend:** PHP with Laravel Framework
3. **Database:** MySQL
4. **Server:** Apache Server (WAMP)
5. **Platform:** Web-Based Application accessible through Internet Browsers (Chrome, Firefox, Edge, etc.)

4.7 Database Requirement

- **Users**
- **Student Profile**
- **Room and Categories**
- **Room Allocation**
- **Complaints**
- **Inventory**
- **Payments**
- **Room Facilities**

4.8 Actor Identification

The following actors interact with the Hostel Management System:

- **employee:** Manages hostel operations including room allocation, student monitoring, complaint management, hostel records, and hostel discipline.
- **Student:** Registers in the system, applies for room allocation, submits complaints, pays hostel fees, and communicates with the employee.

4.9 Database Schema

- **Users Table (Handles 'Login' Use Case):**
  - user_id (PK)
  - name
  - email (Unique)
  - password
  - role (Student / employee)
  - phone_number
  - status
- **Student Table (Linked with 'Manage Students and Allocations'):**
  - student_id (PK)
  - user_id (FK)
  - registration_number
  - department
  - semester
  - address
- **Hostel Table (Linked with 'View Hostel Details' & 'Manage Hostel Inventory'):**
  - hostel_id (PK)
  - hostel_name
  - hostel_type (Boys / Girls)
  - location
  - total_rooms
- **Room Table (Linked with 'Manage Hostel Inventory'):**
  - room_id (PK)
  - hostel_id (FK)
  - room_number
  - room_type
  - capacity
  - occupied_beds
  - status (Available / Full)
- **Booking Table (Covers 'Book Room' & 'Manage Room Booking'):**
  - booking_id (PK)
  - student_id (FK)

  - room_id (FK)
  - booking_date
  - approval_status (Pending / Approved / Rejected)

- **Fee Table (Covers 'Make Payments' & 'Manage Accounts and Fee Records'):**
  - fee_id (PK)
  - student_id (FK)
  - booking_id (FK)
  - amount
  - payment_status (Paid / Unpaid / Pending)
  - due_date
  - payment_date

- **Complaint Table (Covers 'Submit Complaint' & 'Manage Complaint'):**
  - complaint_id (PK)
  - student_id (FK)
  - complaint_type (Room / Mess / Internet / Maintenance)
  - description
  - complaint_status (Pending / In-Progress / Resolved)
  - complaint_date

Chapter 5

System Analysis and Design

5.1 Gantt Chart

The project schedule and timeline for the development of the Hostel Management System are illustrated in Table 5.1. The schedule spans over a period of 7 weeks, mapping out each phase from initial requirements gathering to final testing and documentation.

Table 5.1: Project Gantt Chart

Task / Phase	**Week 1**	**Week 2**	**Week 3**	**Week 4**	**Week 5**	**Week 6**	**Week 7**
Preparation and Setup	X						
Requirement Analysis		X	X				
System Design			X	X			
Module Development				X	X	X	
Integration					X	X	
Testing & Debugging						X	X
Documentation							X

5.2 Process Model and System Design Approach

The development of the Hostel Management System will start by understanding the requirements of students and management employees. After collecting the requirements, the system design, database design, and interface design will be prepared.

The system will be developed in modules. These modules include login management, account management, student management, room management, facility management, inventory management, complaint management, and payment management. Each module will be developed and tested separately to make sure it works properly.

The system will use a layered architecture. The user interface layer will be developed using HTML, CSS, and JavaScript to provide a responsive front-end. The application layer will be developed using PHP to handle backend system functions and logical processes. The database layer will use MySQL to store all system data like user credentials, student profiles, room allocations, facilities, inventory logs, complaints, and fee payments.

After development, the system will be tested to check that all features are working correctly. This approach will help in building a secure and user-friendly Hostel Management System.

5.3 Use Case Diagram

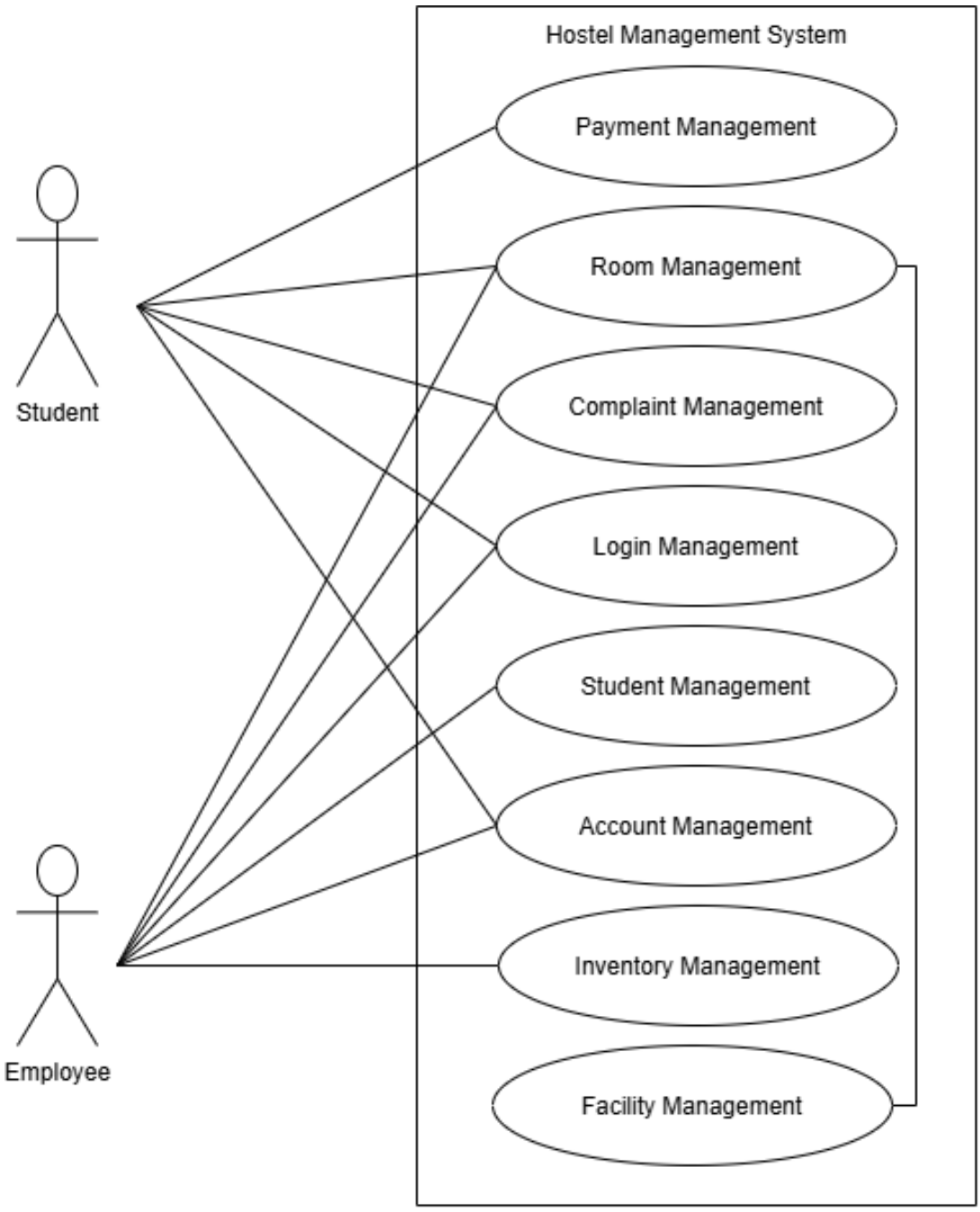


Figure 5.1: Hostel Management System

5.4 Fully-Dressed Use Cases

Use Case Id:	UC-HMS-01
Use Case Name:	Register Account
Actor(s):	Student, Employee
Precondition(s):	User is not registered in the system.
Main Success Scenario:	
01	User opens registration page.
02	System displays registration form.
03	User enters required details (Name, Email, Password, Contact, and Role).
04	System validates the information.
05	System creates a new account profile.
06	Confirmation message is displayed.
Postcondition(s):	User account is created successfully.
Alternative Flow(s):	If the email already exists, the system displays an error message.
Special Requirement(s):	Passwords must be encrypted before storage.

Table 5.2: Use Case Specification for Register Account

Use Case Id:	UC-HMS-01
Use Case Name:	Login Management
Actor(s):	Student, Employee
Precondition(s):	User has a reguster account
Main Success Scenario:	
01	User opens login page.
02	User enters email and password
03	System valid credentials (Name, Email, Password, Contact, and Role).
04	System grants access .
05	Dashboard is displayed
Postcondition(s):	User logs into the system successfully.
Alternative Flow(s):	Invalid credentials displays an error message.
Special Requirement(s):	Session timeout after inactivity.

Table 5.3: Use Case Specification for Login Management

Use Case Id:	UC-HMS-03
Use Case Name:	Room Management
Actor(s):	Student, employee
Precondition(s):	Hostel rooms, bed capacities, and floor layout data must be active inside the master database.
Main Success Scenario:	
01	Student views available vacant slots and rooms categorized by floor maps.
02	Student selects a valid room number and clicks on "Submit Booking Request".
03	System updates the tracking status to "Pending" and forwards it to the employee.
04	employee reviews the student's booking application profile metrics.
05	employee clicks "Approve" to allocate the room assets permanently.
06	System dynamically reduces available bed count and sends a push alert to the Student.
Postcondition(s):	Student profile is linked to the room ID, and inventory tracking counts decrease.
Alternative Flow(s):	If the selected room fills up concurrently before submission, the system displays a "Room Full" error message.
Special Requirement(s):	Concurrency locks must ensure a single room slot cannot be assigned to two tokens simultaneously.

Table 5.4: Use Case Specification for Room Management

Use Case Id:	UC-HMS-04
Use Case Name:	Complaint Management
Actor(s):	Student, employee
Precondition(s):	Student must be currently assigned to a valid room inside the database cluster.
Main Success Scenario:	
01	Student opens the smart complaint portal, selects a category (Plumbing, Wi-Fi, Electrical), and submits the issue text.
02	System saves the entry, logs an auto-generated token ID as "Open", and forwards it to the employee's dashboard.
03	employee reviews the open ticket, assigns a technical specialist, and logs resolution notes after maintenance is performed.
04	employee updates the status to "Resolved", and the system triggers an immediate notification to the student feedback channel.
Postcondition(s):	Grievance record status changes to closed, creating a permanent resolution timestamp log.
Alternative Flow(s):	If the complaint stays unassigned for more than 48 hours, the system executes automatic rule escalations, routing critical tags to higher management views.
Special Requirement(s):	Support data storage must allow students to upload structural reference photos up to 5MB.

Table 5.5: Use Case Specification for Complaint Management

Use Case Id:	UC-HMS-05
Use Case Name:	Payment Management
Actor(s):	Student, employee
Precondition(s):	Student must have an outstanding fee dues record logged, and a verified payment gateway integration must be operational.
Main Success Scenario:	
01	Student navigates to the billing portal where the system displays pending dues, fine breakdowns, and base hostel fees.
02	Student selects a payment method (Credit Card/Digital Wallet), enters the required billing details, and submits the transaction.
03	System securely processes the payment through the gateway API, generates a digital receipt, and updates the student's balance to zero.
04	employee views the real-time updated clearance statement on the admin portal dashboard to confirm student status.
Postcondition(s):	Transaction ledger is cryptographically updated, and payment status changes to "Paid".
Alternative Flow(s):	If the transaction is declined by the bank, the system alerts the student with the error reason and unlocks the invoice for a re-attempt.
Special Requirement(s):	Gateway compliance must strictly adhere to PCI-DSS transaction security standards.

Table 5.6: Use Case Specification for Payment Management

Use Case Id:	UC-HMS-06
Use Case Name:	Student Management
Actor(s):	Student, employee
Precondition(s):	User identity tracking details must pass internal institutional baseline verification steps.
Main Success Scenario:	
01	employee opens the student directory database tools to review overall active check-in indexes and operational logs.
02	Student accesses the personal profile settings panel to update emergency contact details and personal records.
03	System reviews the submitted format structures against validation constraints and flags any inconsistent field inputs.
04	employee reviews and confirms the profile modifications, permanently updating the active synchronization modules.
Postcondition(s):	Core user index records are updated, keeping historical values secure inside metadata logs.
Alternative Flow(s):	If the student attempts to modify fixed administrative indices (e.g., Roll No), the system raises a field block policy alert and freezes programmatic editing pathways.
Special Requirement(s):	Critical personal file updates must trigger an automated email confirmation to registered guardian accounts.

Table 5.7: Use Case Specification for Student Management

Use Case Id:	UC-HMS-07
Use Case Name:	Account Management
Actor(s):	Student, employee
Precondition(s):	Active target account objects must be initialized within core domain tables.
Main Success Scenario:	
01	User navigates to Account Profile and Security Settings configurations to trigger credential updates or request a password modification.
02	System prompts for the active old password to verify account ownership before allowing changes.
03	User supplies the matching current key along with compliant new passphrase strings.
04	System processes background changes, updates password salts, and revokes all other active open sessions.
Postcondition(s):	Account authentication parameters are successfully changed and synced system-wide.
Alternative Flow(s):	If the password complexity rules are unmet, the system rejects changes and displays specific requirements (minimum 8 characters, numbers, and special symbols).
Special Requirement(s):	Passwords must be dynamically encrypted using secure industry hashing standard algorithms (e.g., bcrypt).

Table 5.8: Use Case Specification for Account Management

Use Case Id:	UC-HMS-08
Use Case Name:	Inventory Management
Actor(s):	employee
Precondition(s):	employee must be authenticated; architectural hostel item categories must exist.
Main Success Scenario:	
01	employee accesses the master Inventory and Asset tracking system modules where the system returns stock balances for structural furniture, utility units, and mess items.
02	employee registers incoming asset items or adjusts allocation values for specific floors.
03	employee modifies the operational usage state of specific hardware units (e.g., marking item as broken).
04	System logs changes, alters baseline availability flags, updates resource totals, and syncs data across modules.
Postcondition(s):	Logistics and asset inventory records are modified and matched against room supply schemas.
Alternative Flow(s):	If the stock quantity drops below critical item threshold levels, the system highlights rows in yellow and creates automated restocking alert indicators.
Special Requirement(s):	Updates to physical asset items must mirror instantly onto room repair systems.

Table 5.9: Use Case Specification for Inventory Management

Use Case Id:	UC-HMS-09
Use Case Name:	Facility Management
Actor(s):	Student, employee
Precondition(s):	Core shared facilities (Mess, Gym, Laundry) must be actively initialized in the system database.
Main Success Scenario:	
01	Student or employee checks the active operational tracking status of a specific facility where the system returns real-time parameters, schedules, and active availability flags.
02	employee alters configuration values (e.g., updating mess hours or pausing laundry service).
03	System processes updates, flags structural changes, and updates database records.
04	System pushes notification cards out directly onto all student portal dashboard grids.
Postcondition(s):	Facility operational rules and structural availability matrices are successfully updated.
Alternative Flow(s):	If the employee marks the facility status as "Emergency Breakdown", the system locks booking options, turns visibility tags to red, and runs automated mass alert notifications.
Special Requirement(s):	Status changes made by administration must populate onto student user interfaces within 5 seconds.

Table 5.10: Use Case Specification for Facility Management

5.5 Module Use Case Diagram

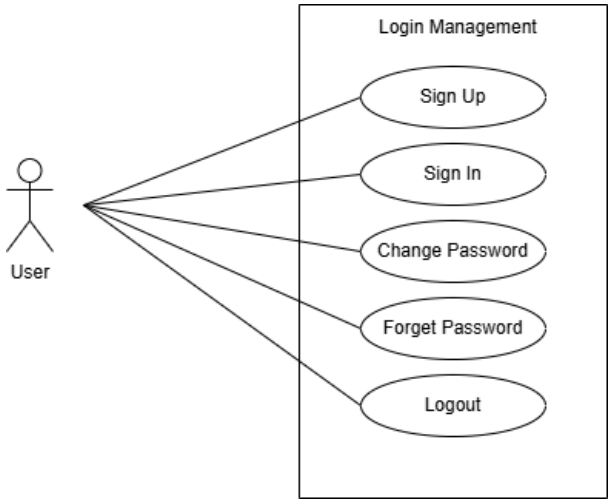


Figure 5.2: Login Management

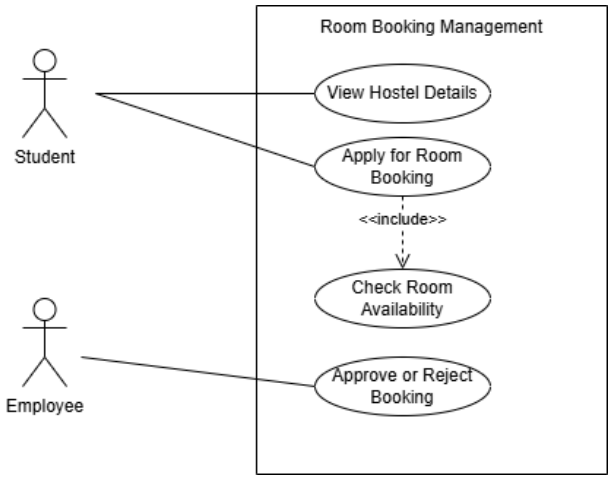


Figure 5.3: Room Management

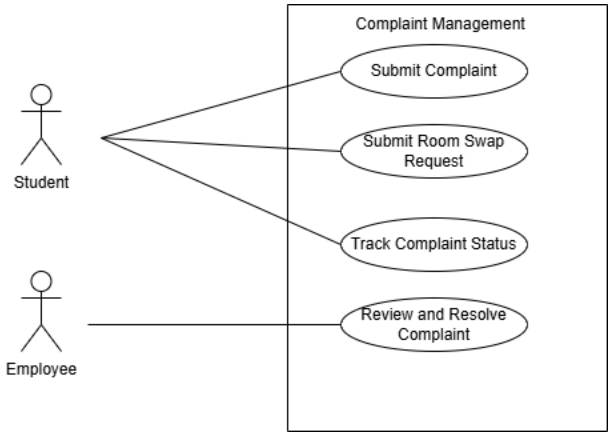


Figure 5.4: Complaint Management

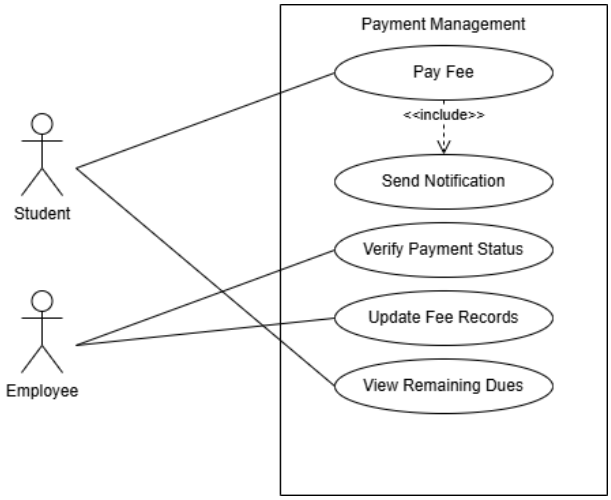


Figure 5.5: Payment Management

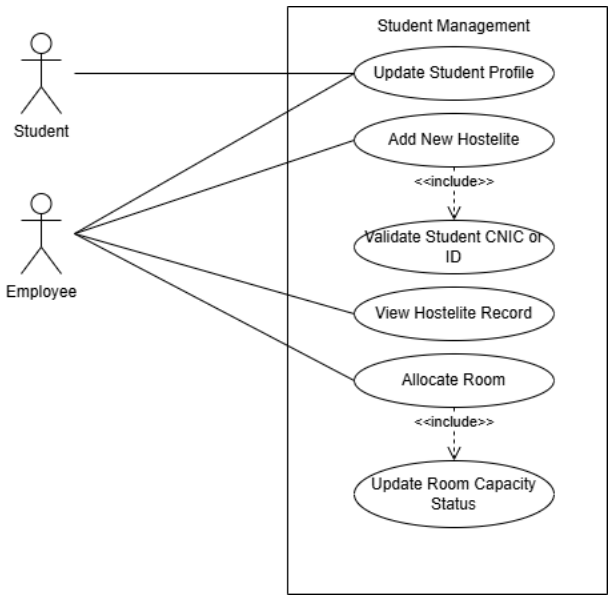


Figure 5.6: Student Management

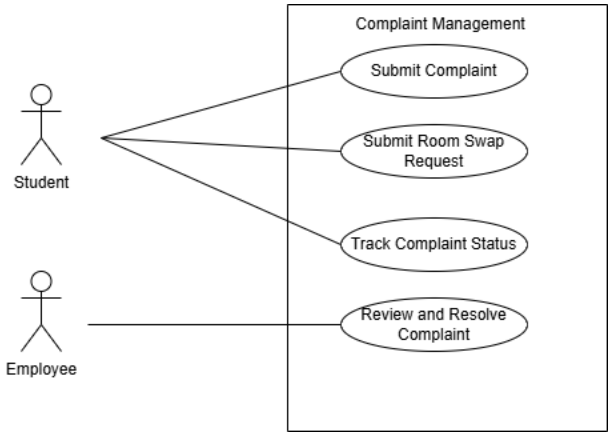


Figure 5.7: Complaint Management

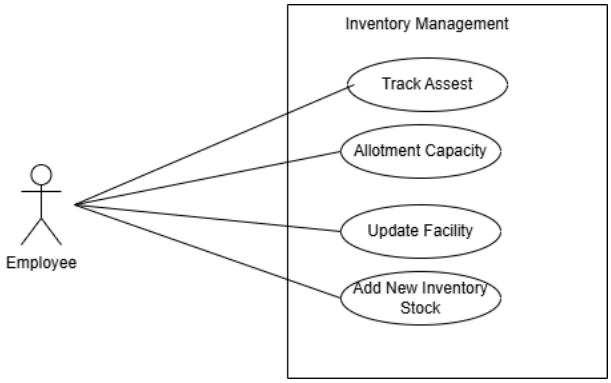


Figure 5.8: Inventory Management

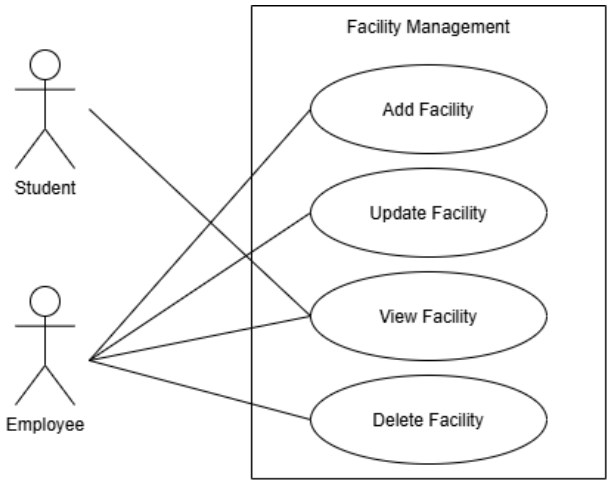


Figure 5.9: Facility Managements

5.6 Class Diagram

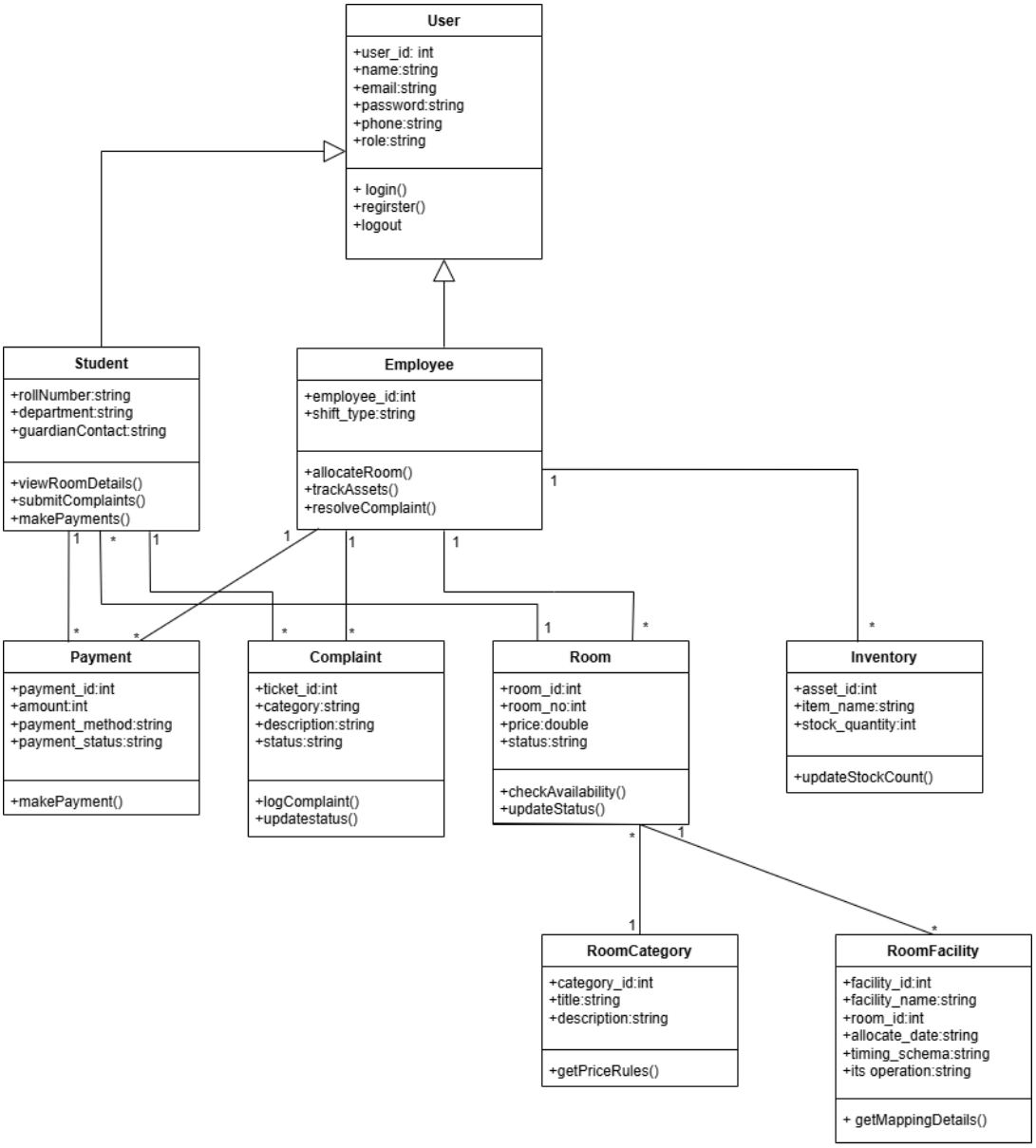


Figure 5.10: Class Diagram

5.7 Sequence Diagram

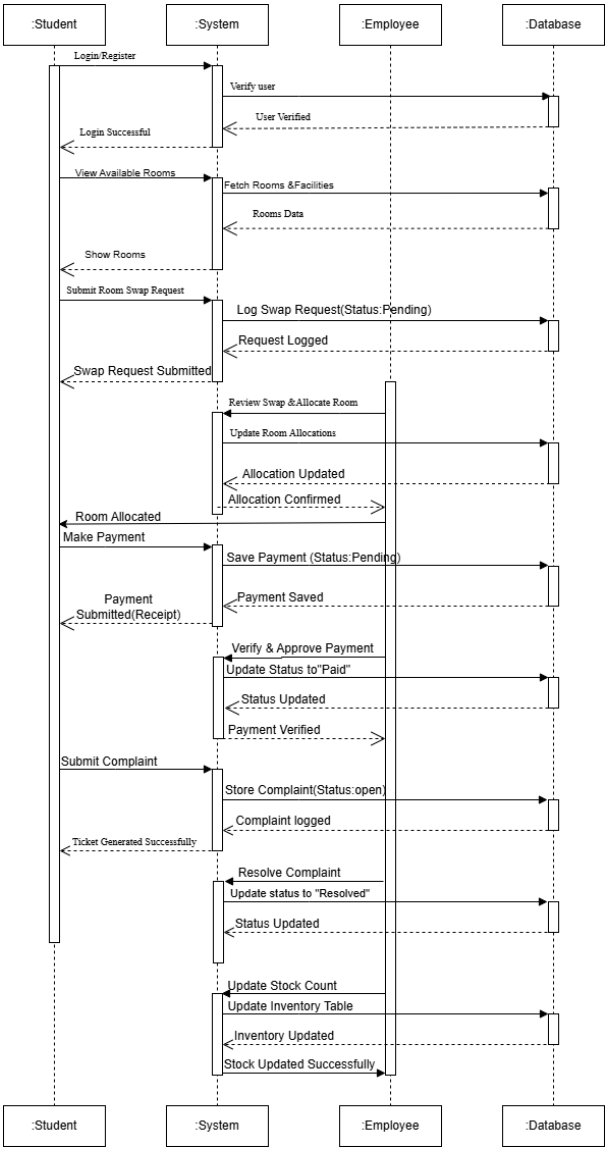


Figure 5.11: Sequence Diagram

5.8 ER Diagram

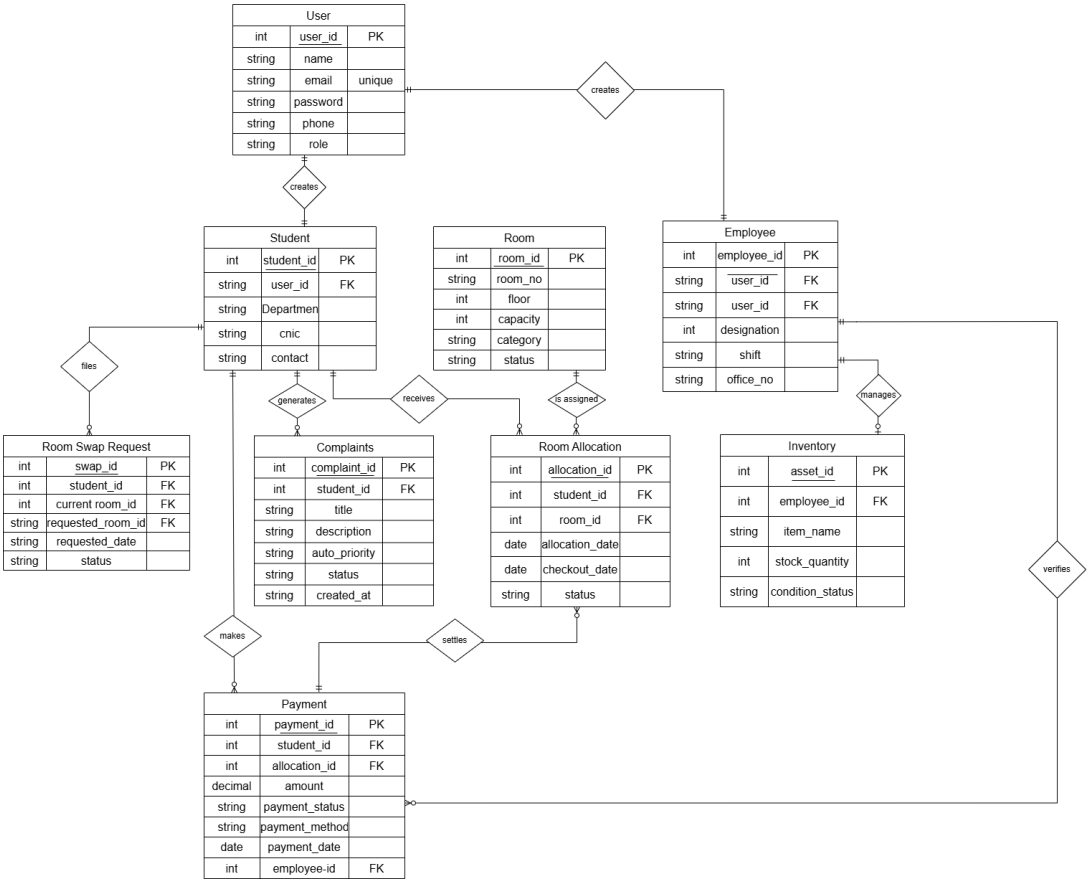


Figure 5.12: ER Diagram

5.9 Domain Model

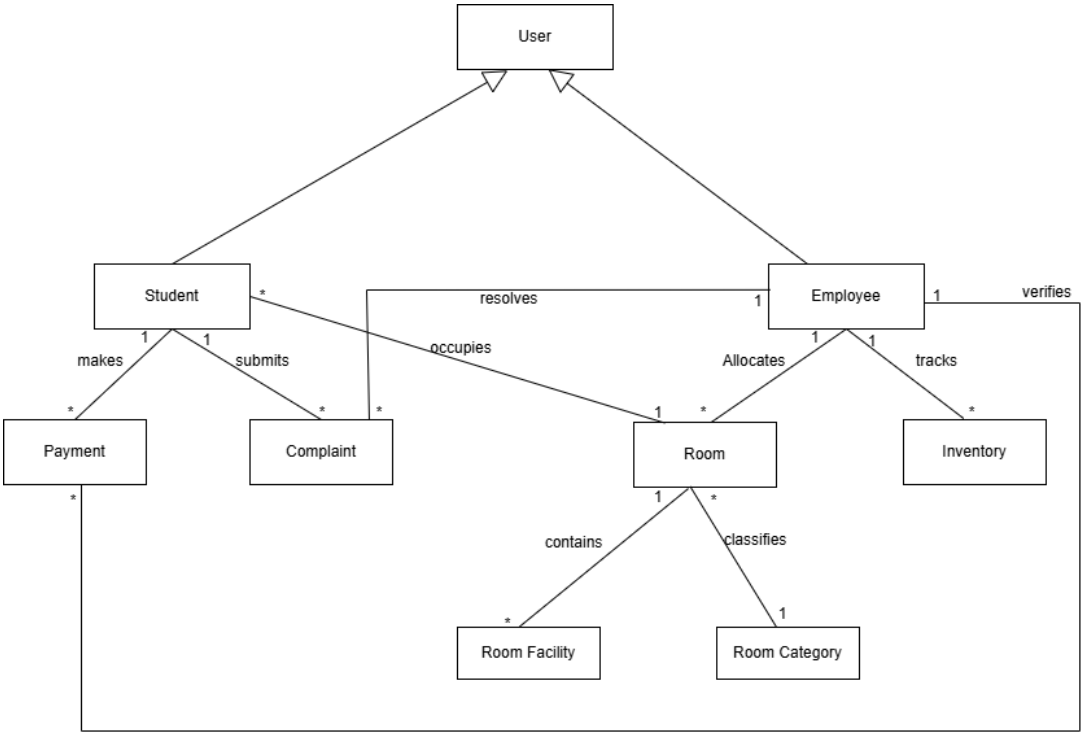


Figure 5.13: Domain Model

5.10 Collaboration Diagram

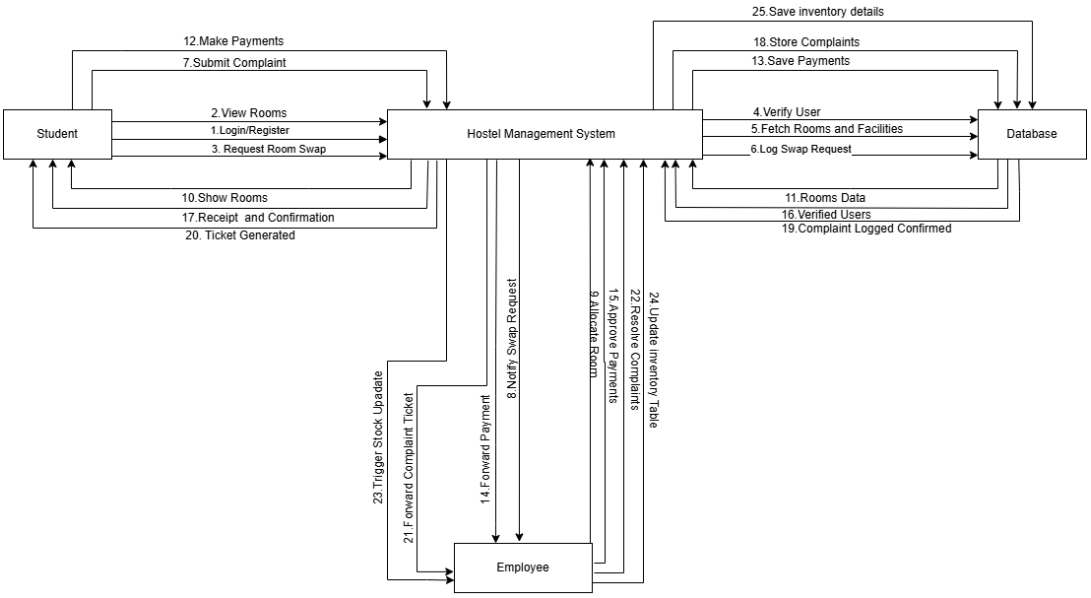


Figure 5.14: Collaboration Diagram

5.11 Activity Diagram

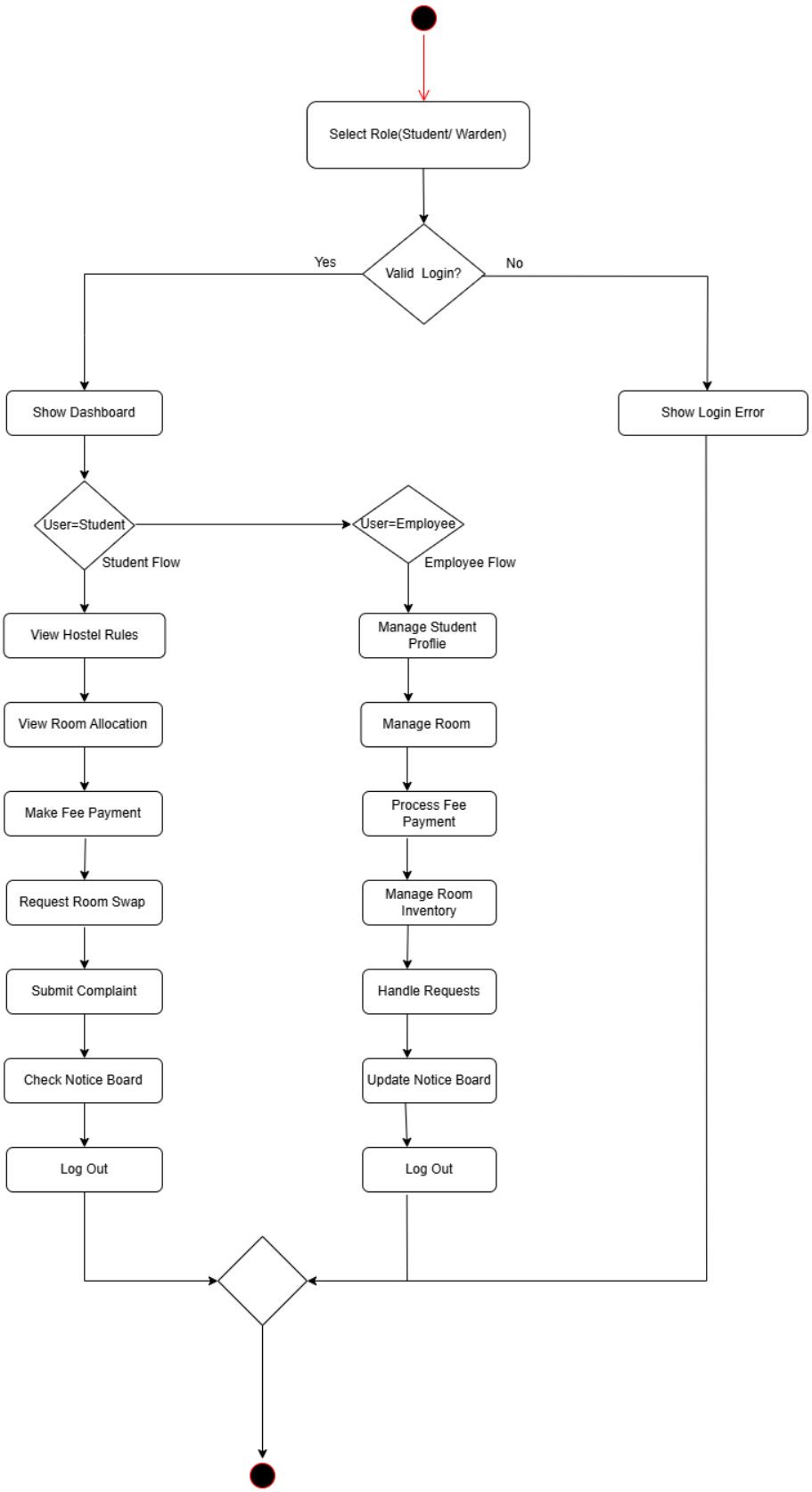


Figure 5.15: Activity Diagram

Chapter 6

Implementation

6.1 Development Tools

- **Front-end:** HTML, CSS, JavaScript
- **Backend:** PHP
- **Database:** MySQL
- **Deployment Tools:** XAMPP / WAMP / Live Server

6.2 Front End Implementation

- **HTML, CSS, JavaScript:** These technologies are used for developing the front-end of the Hostel Management System. HTML provides the structure of web pages, CSS is used for designing and styling, while JavaScript adds interactivity and dynamic functionality to the system.

6.3 Back End Implementation

- **PHP** PHP is used for backend development and business logic implementation. It manages hostel operations such as room allocation, complaint handling, booking management, and fee processing.
- **MySQL Database:** MySQL is used as the relational database management system. It stores hostel records, student data, room details, complaints, fee records, and booking information securely and efficiently.

6.4 Database Implementation

MySQL will be used to implement the database of the Hostel Management System. It will store and organize all important data including student profiles, employee information, room records, booking details, complaints, fee records, facilities information, and hostel requests. The database will ensure data integrity, security, and fast retrieval of information for efficient hostel management operations.

Chapter 7

Future Work and Conclusion

7.1 Future Enhancements

- Mobile Application
- Multi-language support

7.2 Conclusion

The System will be developed to improve hostel services and make hostel services easier to access and understand. The system will help students and employees manage room allocation, tracking fee payments, manage inventory and handling student complaints in a secure and organized way. Overall, the system will improve efficiency, accuracy, and accessibility for both students residing and employees managing the facility.